annotates the dataset we send so that individual datapoints can be drawn as circles or rectangles. A grid isn't the most useful or sexy layout, but it can teach you the basics of layouts. After that, we'll build a legend component that tells users the meaning of the color of our elements. We'll do this by basing the graphical components of the legend on the scale we've used to color our chart elements.

10.1 Creating a layout

Recall from chapter 5 that a layout is a function and an object that modifies a dataset for graphical representation. Here, we'll build that function. Later, we'll give it the capacity to modify the settings of the layout in the same manner that built-in D3 layouts operate.

You'll see this in more detail later, but first we need to create an object that returns the function that processes our data. After we create this function, we'll use it to implement the calls that a layout needs. In the following listing, you can see the function and a test where we instantiate it and pass it data.

Listing 10.1 d3.layout.grid.js

```
d3.layout.grid = function() {
    function processGrid(data) {
        console.log(data)
    }
    return processGrid;
}

var grid = d3.layout.grid();
grid([1,2,3,4,5]);
```

Prints [1,2,3,4,5] to the console

That's not an exciting layout, but it works. We don't need to name our layout `d3.layout.x` or any other particular name, but using that namespace makes it more readable in the future. Before we start working on the functions that will create our grid, we have to define what this layout does. We know we want to put the data on a grid, but what else do we want? Here's a simple spec:

- We want to have a default arrangement of that grid, say, equal numbers of rows and columns.
- We also want to let the user define the number of rows or columns.
- We want the grid to be laid out over a certain size.
- We also need to allow the user to define the size of the grid.

That's a good start. First we need to initialize all the variables that this grid needs to access to make it happen. We also need to define getter and setter functions to let the user access those variables, because we want to keep them scoped to the `d3.layout.grid` function. The first thing we can do is update the `processGrid` function to look like it does in listing 10.2. It takes an array of objects and updates them with x and y data based on grid positions. We derive the size of the grid from the number of data